

PLAN D'IMPLÉMENTATION — MVP 30 JOURS

Plateforme IA Prête à l'Emploi

Stack 100% Free / Open Source & Scalable — Exécution en binôme
(Porteur de projet technique + Développeur Fullstack)

Document de cadrage technique & opérationnel

Basé sur l'Analyse Stratégique, le Catalogue (1 030 fonctions / 22 catégories) et la Suite ICES Digital
Juin 2026

1. De quoi parle ce projet ?

Le projet vise à construire une **plateforme web d'outils IA "prêts à l'emploi"**, multilingue et destinée à un usage professionnel large (PME, freelances, grandes entreprises, ONG, institutions). Trois documents définissent l'ambition :

- **Analyse stratégique** : positionnement face à Zapier, Make, Notion AI, HubSpot et Microsoft 365 Copilot ; architecture microservices cloud-native ; vision à 5M utilisateurs et 65M€ d'ARR à horizon 5 ans.
- **Catalogue de la plateforme** : **1 030 fonctions IA réparties en 22 catégories** (diagnostics, audits, RH, finance, stratégie, marketing, juridique, gestion de projet, RSE, etc.), couvrant 212 secteurs et prévues en 20 langues.
- **Suite ICES Digital** : des outils d'auto-évaluation/certification (ISO 9001, 14001, 45001, 27001, RSE, bailleurs de fonds...) avec scoring, radar de maturité et génération de rapport PDF.

En clair : **un hub unique qui regroupe des centaines de mini-outils IA** (chacun = un formulaire + un prompt + une réponse structurée), packagés dans une interface cohérente, multilingue, avec abonnement et tableau de bord.

Écart de réalisme à noter

Le document stratégique projette une équipe de 8 à 25 personnes sur 24 mois avec une stack "Enterprise" (Kubernetes, Kafka, multi-cloud AWS+GCP, Datadog...). Ce plan ci-dessous est une **réécriture réaliste pour 2 personnes sur 30 jours** : même vision produit, mais stack 100% free/open source et architecture pensée pour scaler progressivement plutôt que sur-ingénierée dès le jour 1.

2. La clé pour viser "le maximum d'outils possible"

Construire 1 030 outils IA "à la main" (1 page de code par outil) est impossible en un mois. La seule façon réaliste d'en couvrir un grand nombre rapidement est d'adopter une **architecture "moteur générique piloté par configuration"** :

Principe : on ne code *qu'une seule fois* le moteur (formulaire dynamique → prompt → appel LLM → rendu de sortie + export PDF). Chaque "outil" devient ensuite une simple **entrée de configuration JSON** (titre, catégorie, champs du formulaire, template de prompt, format de sortie attendu) — pas du code. Ajouter un outil = ajouter une fiche de config, pas développer une feature.

Élément	Approche classique (1 dev / outil)	Approche "moteur générique" retenue
Temps pour 100 outils	Plusieurs mois	Quelques jours (génération assistée par IA des configs)
Maintenance	Code dupliqué partout	1 seul moteur à maintenir
Comment "écrire" un outil	Code React + route API dédiée	Fiche JSON : champs + prompt template
Qui peut créer un outil	Développeur uniquement	Le porteur de projet (toi) aussi, sans coder

Concrètement : on demande à une IA (Claude, GPT-4o-mini, etc.) de transformer chaque ligne du catalogue (1 030 fonctions) en fiche JSON structurée (titre + 3-6 champs d'input + prompt système). C'est un travail de génération en masse, supervisé, pas de développement. C'est ce qui rend "un maximum d'outils en 1 mois" réaliste.

3. Stack technologique recommandée (Free / Open Source / Scalable)

Priorité : zéro coût fixe au démarrage, tout sur des offres "free tier" généreuses et open source, avec un chemin de scale clair (pas besoin de tout réécrire si la plateforme grossit).

Couche	Choix recommandé	Pourquoi	Coût
Frontend	Next.js 14 (React) + TypeScript + Tailwind CSS	Open source, SSR/SEO, écosystème énorme, parfait pour formulaires dynamiques	Gratuit
Hébergement frontend	Vercel (free tier)	Déploiement Next.js natif, CDN inclus, scale auto	Gratuit puis usage
Backend / API	Next.js API Routes (puis extraction Fastify/Node si besoin)	Un seul repo au début = vitesse d'exécution pour 2 personnes	Gratuit
Base de données	PostgreSQL via Supabase (free tier) ou Neon	Open source, relations + JSON hybride, Auth & Storage inclus avec Supabase	Gratuit (500MB-1GB)
Authentification	Supabase Auth (ou NextAuth.js)	OAuth2, magic link, gratuit, évite de coder l'auth	Gratuit
Moteur IA / LLM	Routeur multi-LLM : OpenRouter (accès à de nombreux modèles, dont des gratuits) + fallback Groq (free tier très rapide) + option Ollama (modèles open source auto-hébergés) pour les volumes élevés	Pas de lock-in fournisseur, coûts variables maîtrisés, certains modèles gratuits	Free tier + pay-as-you-go
Stockage fichiers / PDF générés	Cloudflare R2 (free tier 10GB) ou Supabase Storage	Pas de frais de sortie (egress) chez Cloudflare	Gratuit
Génération PDF (rapports)	Bibliothèque open source : react-pdf ou Puppeteer/Playwright (HTML→PDF)	Identique aux rapports ICES / radar de maturité	Gratuit
i18n / multilingue	next-intl ou i18next	Standard open source, lazy loading par langue	Gratuit
Paiement / abonnement (international)	Stripe (mode test gratuit, commission seulement sur transaction réelle)	Standard du marché pour cartes bancaires internationales, pas de coût fixe	Gratuit en test
Agrégateur de paiement (Afrique / Mobile Money)	CinetPay ou FedaPay ou Kkiapay (au choix) — Mobile Money (MTN, Moov, Orange) + cartes locales	Stripe ne couvre pas le Mobile Money ouest-africain ; ces agrégateurs sont nés localement, intégration simple (API + webhook), pas d'abonnement fixe	Gratuit setup, commission/transaction
CI/CD	GitHub Actions (free tier) + déploiement auto Vercel	Intégré, gratuit pour repo privé limité	Gratuit
Monitoring / erreurs	Sentry (free tier) + Vercel Analytics	Suffisant pour un MVP	Gratuit
Référentiels normatifs (ISO, RSE...)	Table PostgreSQL dédiée (referentiels + clauses), pgvector si recherche sémantique nécessaire plus tard	Évite les hallucinations sur les normes, versionnable sans redéploiement	Gratuit
Validation & qualité des outils	Zod (validation de schéma) + JSON Schema + script de test automatisé + LLM "juge"	Open source, garantit la fiabilité des outils générés en masse (cf. section 4.3)	Gratuit
Export outils hors-ligne	esbuild (bundling HTML autonome) ; WebLLM (roadmap V2, IA 100% navigateur)	Open source, permet l'usage des outils sans connexion à la plateforme	Gratuit
Design system / UI Kit	shadcn/ui + Tailwind (composants open source copiables)	Va vite, look professionnel, pas de licence	Gratuit

Seule dépense quasi-inévitable : l'usage des API LLM (au-delà des free tiers) si le volume de générations explose.
Stratégie : démarrer avec les options gratuites/à très faible coût (Groq, modèles gratuits sur OpenRouter, Gemini Flash free tier) et plafonner l'usage par utilisateur (quotas) dès le MVP pour garder le coût proche de 0€ pendant la phase de test.

Choix de l'agrégateur de paiement local

Pour encaisser en Mobile Money (MTN, Moov, Orange Money) et cartes locales en Afrique de l'Ouest/Centrale, en complément de Stripe pour les paiements internationaux par carte :

Agrégateur	Couverture	Points forts	Intégration
FedaPay	Bénin, Côte d'Ivoire, Togo, Sénégal...	Né au Bénin, support local réactif, SDK simple, mode test gratuit	API REST + SDK Node.js/JS, webhooks
Kkiapay	Bénin, zone UEMOA	Très simple à intégrer (widget JS), pensé pour les développeurs solo/startups	Widget JS + API REST
CinetPay	UEMOA + Afrique Centrale, large	Couverture géographique la plus large, fiable, utilisé par de nombreuses plateformes SaaS	API REST + checkout hébergé
PayDunya /	Alternatives	Bonnes alternatives si besoin d'une couverture pays plus large	API REST

Recommandation MVP : démarrer avec **FedaPay** ou **Kkiapay** (les deux sont nés au Bénin, intégration rapide, pas de frais fixes, parfait pour un déploiement en 30 jours), et garder Stripe en parallèle pour les clients payant par carte internationale. Architecture : un module de paiement abstrait (interface commune) dans le code, avec les deux fournisseurs branchés derrière — facilite l'ajout d'un 3^e agrégateur plus tard sans tout refaire.

Chemin d'évolution (scalabilité) sans réécriture

Palier	Déclencheur	Action
0 → 5 000 utilisateurs	MVP / beta	Stack ci-dessus telle quelle (tout free tier)
5 000 → 50 000	Limites Supabase free atteintes	Upgrade Supabase Pro (~25\$/mois) ou migration Postgres managé (Neon/RDS)
50 000+	Besoin d'isolation / SLA	Extraction des routes API critiques en microservices (Fastify), passage Kubernetes si nécessaire — l'architecture modulaire dès le départ permet cette extraction sans tout casser

4. Exemple concret d'un outil & système de vérification qualité

4.1 Ce qu'un outil fait concrètement, exemple

Prenons "**Diagnostic de maturité digitale (DMI)**" du catalogue. Côté utilisateur : un formulaire (secteur d'activité, taille d'équipe, outils actuellement utilisés, budget IT) → bouton "Générer" → résultat structuré : un score global, un score par axe (infrastructure, compétences, processus, culture), un radar visuel, et 3 à 5 recommandations priorisées, exportables en PDF.

4.2 Un JSON seul suffit-il ?

Non, pas seul. Le JSON est la **donnée** (la configuration de l'outil) : titre, champs du formulaire, template de prompt, schéma de sortie attendu. Il a besoin d'un **moteur** (code construit une seule fois par le développeur) qui le lit et l'exécute.

Exemple de fiche JSON pour le DMI :

```
{
  "id": "dmi-001",
  "title": "Diagnostic de maturité digitale",
  "category": "diagnostics",
  "inputs": [
    { "name": "secteur", "type": "select", "options": ["Industrie","Services","Commerce","ONG"] },
    { "name": "taille_equipe", "type": "number" },
    { "name": "outils_actuels", "type": "textarea" }
  ],
  "promptTemplate": "Tu es consultant en transformation digitale. Secteur: {secteur}. Taille équipe: {taille_equipe}. Outils actuels: {outils_actuels}. Évalue la maturité sur 4 axes...",
  "outputSchema": {
    "type": "object",
    "properties": {
      "score_global": { "type": "number" },
      "axes": { "type": "array", "items": { "axe": "string", "score": "number" } },
      "recommandations": { "type": "array", "items": "string" }
    }
  }
}
```

Le moteur fait : lire les inputs → afficher le formulaire → injecter les réponses dans promptTemplate → appeler le LLM → vérifier que la réponse respecte outputSchema → afficher le radar + générer le PDF.

Le piège à grande échelle : si le LLM répond en texte libre au lieu de respecter un format précis, l'affichage casse (pas de score à mettre dans le radar, pas de structure pour le PDF). À 1 000 outils générés en masse, sans garde-fous, on obtient des fiches JSON "jolies" mais une part significative d'outils cassés ou médiocres en pratique. D'où la nécessité d'un système de vérification robuste.

4.3 Système de vérification robuste — 4 niveaux

Niveau	Quand	Ce qui est vérifié	Outil / méthode
1. Validation structurelle	Avant publication de la fiche	Champs obligatoires présents, types valides, les variables du prompt ({secteur}...) correspondent bien aux inputs déclarés	Schéma de validation Zod (Node.js) ou JSON Schema — rejet automatique si non conforme
2. Contrat de sortie forcé	À chaque appel LLM	La réponse du LLM doit respecter exactement outputSchema (pas de texte libre)	Mode "structured output / JSON mode" natif des API LLM (OpenAI, Anthropic, etc.) + 1-2 retries automatiques avec l'erreur de validation réinjectée dans le prompt si non conforme
3. Banc de tests automatisé	Avant publication d'un outil (statut draft → published)	Exécution de bout en bout sur 2-3 jeux d'inputs réalistes générés automatiquement ; vérification du schéma, absence de champs vides/absurdes, temps de réponse ; notation qualité 1-5 par un LLM "juge" (pertinence, cohérence, actionnabilité)	Script de test automatisé + LLM juge — sous le seuil de qualité, l'outil reste invisible aux utilisateurs
4. Supervision en production	En continu après mise en ligne	Retour réel des utilisateurs (👍/👎 sur chaque résultat), taux d'erreurs de schéma, taux de notes négatives par outil	Tableau de bord qualité ; tout outil hors clous repasse automatiquement en draft pour relecture

À retenir : JSON = matière première. Ce qui rend un outil réellement "correct" et publiable, c'est la combinaison validation de structure + sortie LLM contrainte par schéma + tests automatisés avant publication + retour terrain en continu. C'est ce filtre à 4 niveaux qui permet de viser "le maximum d'outils possible" sans publier des centaines d'outils de qualité aléatoire.

5. Référentiels normatifs & téléchargement des documents

5.1 Intégrer les normes internationales (ISO 9001, 14001, 45001, 27001, RSE...) sans risque

Laisser le LLM "réciter de mémoire" une norme est risqué : confusion possible entre versions (ex. ISO 27001:2013 vs 2022), réponses imprécises, hors-sujet. La bonne pratique est de séparer le **référentiel** (la connaissance normative) du **prompt**, et d'injecter au moteur uniquement les clauses pertinentes au moment de l'exécution — plutôt que de compter sur la mémoire interne du modèle.

Élément	Description
Table référentiels	Une norme = une fiche (ex. "ISO 9001:2015") versionnée, modifiable depuis le back-office sans redéploiement
Table clauses	Chaque norme décomposée en clauses (ex. "4.1 Compréhension du contexte") avec une exigence reformulée par l'équipe + grille de notation (0 à 4)
Injection au prompt	L'outil ICES reçoit en contexte les clauses précises du référentiel concerné, pas une consigne vague — élimine la majorité des erreurs/hallucinations normatives
Recherche (si croissance)	10-15 normes x 50-100 clauses tient en filtrage simple au démarrage ; si le volume de référentiels explose, ajout de pgvector (extension gratuite/open source de PostgreSQL, déjà compatible Supabase) pour une recherche sémantique des clauses pertinentes

Point légal important : les textes officiels ISO/IAF sont protégés (payants, droits d'auteur). On ne reproduit jamais le texte officiel — on rédige nos propres formulations de critères d'auto-évaluation, inspirées de la structure publique des normes (c'est déjà l'approche de la Suite ICES dans les documents fournis).

5.1.1 Comment assurer la mise à jour des normes dans le temps

Mécanisme	Détail
Veille normative trimestrielle	Revue manuelle du catalogue officiel ISO/AFNOR pour les normes couvertes — les révisions majeures sont rares (5-10 ans), une revue trimestrielle suffit ; plus fréquent pour les référentiels RSE/bailleurs de fonds
Versioning natif	Une nouvelle version (ex. ISO 27001:2022) est ajoutée à côté de l'ancienne, jamais en écrasant — l'ancienne passe en statut "déprécié" mais reste accessible pour les rapports déjà générés (traçabilité) ; changelog horodaté des modifications
Édition sans redéploiement	Back-office permettant de modifier/ajouter des clauses en quelques minutes, sans toucher au code
Alerte automatique (optionnel)	Script léger et gratuit (GitHub Actions cron) qui détecte les changements sur la page catalogue officielle ISO et envoie une alerte — complément à la veille humaine, ne la remplace pas

5.2 Permettre le téléchargement des rapports générés

Format	Usage	Outil open source
PDF	Rapport mis en forme : logo, en-tête/pied de page, radar de maturité — format principal	Playwright / Puppeteer (HTML→PDF)
DOCX (optionnel)	Pour les utilisateurs qui veulent éditer le rapport eux-mêmes	Librairie docx (npm), même structure que le PDF
XLSX / CSV	Sorties type checklist/scoring avec beaucoup de lignes (audits)	SheetJS (xlsx)
Export groupé (ZIP)	Télécharger tous ses rapports en une fois	Librairie archiver

Flux : génération du résultat structuré → sauvegarde en base → rendu à la demande dans le format choisi → stockage du fichier (Cloudflare R2 / Supabase Storage) → lien de téléchargement listé dans l'historique du tableau de bord utilisateur (cf. section 6), avec renvoi par email en option.

5.3 Téléchargement des outils eux-mêmes en mode autonome hors-ligne

Différent du téléchargement de rapport : ici l'utilisateur télécharge **l'outil lui-même** (le formulaire + sa logique) pour l'utiliser sans être connecté à la plateforme. Le défi technique : la génération IA passe normalement par un appel cloud — sans internet, pas d'appel possible, sauf à embarquer un modèle d'IA dans le fichier (lourd, plus lent, qualité réduite).

Palier	Faisabilité	Fonctionnement
Mois 1 (réaliste)	✔ Inclus au MVP	Export de chaque outil en un seul fichier HTML autonome (formulaire + mise en page + logique, ouvrable directement dans un navigateur, sans serveur). Pour les outils de scoring/audit (type ICES), le calcul est déterministe et tourne 100% hors ligne (aucun appel IA requis). Pour les outils générant du texte, l'utilisateur saisit sa propre clé API (stockée localement) pour la partie IA quand il a une connexion ponctuelle, ou utilise un jeu de recommandations pré-générées par tranche de score (fonctionne sans connexion, moins personnalisé)
V2 (post mois 1)	🚧 Roadmap	Vrai hors-ligne avec IA embarquée via WebLLM (projet open source MLC-AI) : modèle de langage tournant entièrement dans le navigateur via WebGPU, sans serveur. Faisable techniquement mais téléchargement lourd (plusieurs centaines de Mo à quelques Go) et qualité inférieure aux modèles cloud — réservé aux outils les plus simples

Implémentation technique (mois 1) : un script de build (**esbuild**, open source) compile chaque fiche JSON d'outil en un fichier HTML unique auto-suffisant (HTML+CSS+JS inlineés), déclenché depuis le back-office. Le fichier est stocké dans Cloudflare R2 et proposé en téléchargement ("Télécharger la version hors-ligne") depuis la fiche de l'outil et le tableau de bord.

6. Architecture fonctionnelle du MVP

Bloc	Description
Catalogue d'outils	Page de navigation/recherche/filtre par catégorie, alimentée par les fiches JSON des outils
Moteur de formulaire dynamique	Génère automatiquement le formulaire d'un outil à partir de sa fiche JSON (champs texte, select, upload simple)
Moteur de génération IA	Construit le prompt à partir des réponses utilisateur + template, appelle le LLM, structure la réponse
Moteur de scoring (ICES)	Spécifique aux outils d'auto-évaluation : calcule un score par axe à partir des référentiels normatifs (section 5.1), génère un radar (graphique SVG) et un plan d'actions
Générateur de documents téléchargeables	Transforme la sortie IA + le scoring en PDF (par défaut), avec options DOCX et XLSX (section 5.2)
Auth & abonnement	Inscription, plan gratuit avec quota, plan payant via Stripe (carte internationale) + agrégateur local (Fedapay/Kkiapay) pour Mobile Money, module de paiement abstrait commun aux deux
Dashboard utilisateur	Historique des outils utilisés, rapports générés, suivi de progression
Admin / back-office léger	Interface simple pour ajouter/éditer les fiches JSON des outils sans toucher au code

7. Répartition des rôles (binôme)

Rôle	Responsabilités principales
Toi (porteur de projet, technique)	Architecture des prompts, génération assistée par IA des fiches d'outils (configs JSON), contenu, design système (UI), QA fonctionnelle, i18n FR/EN, suivi produit
Développeur Fullstack	Setup infra (repo, CI/CD, Supabase, Vercel), moteur de formulaire dynamique, route API de génération IA, moteur de scoring/PDF, auth, Stripe, déploiement

Les deux travaillent en parallèle dès le jour 1 : le développeur construit le moteur pendant que tu prépares en masse les fiches de configuration des outils (le "carburant" du moteur).

8. Chronogramme d'exécution — 30 jours

4 semaines, 2 personnes en parallèle. Objectif fin de mois : plateforme déployée, multilingue (FR/EN minimum), couvrant **le plus grand nombre d'outils possible** via le moteur générique, prête pour une beta fermée.

Semaine 1 (J1-J7) — Fondations

Jour	Toi	Développeur Fullstack
J1	Cadrage final : choix des 22 catégories à prioriser, structure des fiches JSON (schéma d'un outil)	Setup repo GitHub, projet Next.js + TypeScript + Tailwind, compte Supabase, compte Vercel, CI/CD de base
J2	Rédaction du schéma de prompt-type (template universel par catégorie : diagnostics, rédaction, analyse...) + premier jet des référentiels normatifs prioritaires (ISO 9001, RSE) en clauses reformulées	Modèle de données PostgreSQL (users, tools, submissions, reports, plans, referentiels, clauses)
J3	Test des fournisseurs LLM (OpenRouter, Groq, Gemini) sur 5 prompts pilotes, choix du routeur	Intégration Supabase Auth (inscription/connexion), structure des pages (layout, navigation)
J4	Démarrage génération assistée par IA des fiches JSON (lot 1 : catégories Diagnostics + Templates)	Moteur de formulaire dynamique (rendu d'un outil à partir d'une fiche JSON)
J5	Génération fiches JSON (lot 2)	Route API /api/generate (prompt builder + appel LLM en mode JSON forcé + validation Zod du schéma de sortie + retries automatiques)
J6	Génération fiches JSON (lot 3) + relecture qualité lot 1	Page catalogue (liste/recherche/filtre par catégorie)
J7	Revue binôme : test de bout en bout sur 10 outils, ajustements	Revue binôme : correctifs moteur suite aux tests

Semaine 2 (J8-J14) — Industrialisation du catalogue

Jour	Toi	Développeur Fullstack
J8-J9	Génération fiches JSON en masse (catégories Audits, RH, Finance, Stratégie, Marketing) — objectif 300+ outils	Moteur de rendu de sortie (texte structuré, tableaux, listes) + composant de partage/export
J10-J11	Génération fiches JSON (catégories Juridique, PMO, Innovation, RSE, Risques, Digital) — objectif 600+ outils	Module ICES : moteur de scoring + génération de radar (graphique SVG/Chart.js)
J12-J13	Génération fiches JSON (catégories restantes : Supply Chain, Communication, Qualité, Veille, Sectoriels) — objectif proche de 1 000+ outils bruts ; finalisation des référentiels normatifs restants (14001, 45001, 27001, bailleurs)	Génération PDF (Playwright HTML→PDF) pour rapports d'outils et d'auto-évaluation ICES + options export DOCX/XLSX
J14	Revue binôme : tri qualité, désactivation des fiches faibles, priorisation des 150-200 meilleures pour mise en avant	Revue binôme : mise en place du banc de tests automatisé (niveau 3) — exécution de bout en bout + notation LLM juge sur tout le catalogue généré, passage en statut published uniquement au-dessus du seuil qualité

Pourquoi viser large puis trier : générer massivement les fiches (semaine 2) puis filtrer (qualité, doublons, pertinence) est plus rapide que de vouloir une perfection outil par outil. Le moteur permet d'activer/désactiver un outil en un clic.

Semaine 3 (J15-J21) — Monétisation, i18n & UX

Jour	Toi	Développeur Fullstack
J15-J16	Traduction de l'interface (FR/EN), adaptation des prompts pour réponse bilingue	Intégration Stripe (carte internationale) + intégration FedaPay/Kkiapay (Mobile Money), module de paiement abstrait, gestion des quotas d'usage par plan
J17	Rédaction des pages marketing (accueil, pricing, à propos)	Dashboard utilisateur (historique, rapports générés, abonnement) + script d'export HTML autonome (esbuild) pour le téléchargement hors-ligne des outils de scoring
J18	Mise en avant éditoriale du catalogue (catégories vedettes, outils populaires)	Back-office léger : interface admin pour activer/éditer les fiches outils sans coder
J19	Tests utilisateurs internes (5-10 outils par catégorie testés en conditions réelles)	Corrections suite aux tests, optimisation performance (cache des prompts fréquents)
J20	QA fonctionnelle complète (parcours inscription → outil → rapport → upgrade)	QA technique (sécurité de base : rate limiting, validation des inputs)
J21	Revue binôme : arbitrage des bugs bloquants vs non-bloquants	Revue binôme : idem

Semaine 4 (J22-J30) — Stabilisation & lancement beta

Jour	Toi	Développeur Fullstack
J22-J23	Correction de contenu (prompts, libellés, traductions)	Correction de bugs, durcissement sécurité (variables d'env, secrets, HTTPS)
J24-J25	Préparation du plan de communication beta (liste d'attente, 50-100 testeurs ciblés)	Mise en place monitoring (Sentry) + alertes basiques, sauvegardes DB automatiques

J26	Recette finale complète (check-list de tous les parcours critiques)	Déploiement production sur Vercel + domaine personnalisé
J27	Rédaction documentation utilisateur (FAQ, guide de démarrage)	Tests de charge légers (simulateur de pics d'usage)
J28	Buffer correctifs	Buffer correctifs
J29	Lancement beta fermée (invitations premiers testeurs)	Support technique au lancement, surveillance temps réel
J30	Collecte des premiers retours, premier point de pilotage (NPS, bugs, usage par catégorie d'outil)	Idem + correctifs urgents

9. Risques principaux & parades

Risque	Impact	Parade
Volume de 1 030 outils trop ambitieux pour 1 mois	Qualité hétérogène	Approche "générer large, trier ensuite" + le moteur permet d'activer/désactiver un outil en un clic, sans redéploiement
Coût API LLM qui explose si usage viral	Budget	Quotas stricts par compte gratuit dès le J1, alerte de consommation, modèles gratuits/peu chers en priorité
Qualité des réponses IA inégale selon les prompts générés en masse	Expérience utilisateur	Échantillonnage qualité (1 outil sur 5 testé manuellement avant mise en avant), système de notation utilisateur post-génération
Limites du free tier Supabase/Vercel atteintes plus vite que prévu	Disponibilité	Monitoring des quotas dès la semaine 3, plan d'upgrade payant à faible coût déjà identifié (cf. tableau scalabilité)
Multilingue à 20 langues hors de portée en 1 mois	Promesse non tenue	Démarrer FR/EN uniquement (les prompts génèrent déjà dans la langue demandée par le LLM), élargir progressivement après la beta

10. Après le mois 1

- Élargir à 3-5 langues supplémentaires (ES, PT, AR) une fois le moteur stabilisé.
- Ajouter les modules "Workspace collaboratif" et "Intégrations" (Phase V1.2 du document stratégique original).
- Industrialiser le contrôle qualité des fiches d'outils (revue communautaire ou comité interne).
- Réévaluer le besoin de migrer certaines briques vers une stack plus lourde uniquement si les seuils d'usage du tableau de scalabilité (section 3) sont atteints.

Document généré comme support de pilotage — à ajuster selon la disponibilité réelle du binôme et les premiers retours techniques de la semaine 1.